宁波工程学院

毕业设计（论文）

设计（论文）题目	基于区块链的分布式 AI 数据确权系统设计
学 院	机器人学院
专 业	人工智能 (AI221)
班 级	AI221
姓 名 学 号	王孝萌 22480010119
指 导 教 师	王艳
企 业 导 师	

定稿日期: 2026 年 4 月 20 日

诚 信 承 诺

我谨在此承诺：本人所写的毕业论文《基于区块链的分布式 AI 数据确权系统设计》均系本人独立完成，没有抄袭行为，凡涉及其他作者的观点和材料，均作了注释，若有不实，后果由本人承担。

承诺人（签名）： 王孝萌

2026 年 4 月 20 日

摘 要

近年来人工智能技术迅速发展，AI 模型训练对高质量数据的需求持续增长，而数据的权属界定、流通可信度与溯源能力逐渐成为制约 AI 产业健康发展的关键问题。在中心化数据平台中，数据的复制成本趋近于零，数据提供者的权益难以得到有效保护，数据消费者也缺乏核实数据来源合法性的技术手段。针对这一问题，本文结合区块链的不可篡改性、分布式存储的去中心化特性以及智能合约的自动化执行能力，设计并实现了一套面向 AI 训练数据场景的分布式数据确权系统。

本文的主要工作包括：第一，提出了一种基于 SHA-256 数据指纹与链上时间戳的数据资产唯一标识方案，采用时间优先原则确定数据产权归属，并在批量数据场景下使用 Merkle 树汇聚哈希以支持高效验证；第二，设计了“链上索引 + 链下加密存储”的混合存储架构，链上仅保存数据的哈希、IPFS 内容标识符（CID）与元数据，原始数据经 AES 对称加密后存储于 IPFS，兼顾链上成本与数据安全；第三，基于 Solidity 0.8.20 实现了 DataRegistry 核心确权合约，包含数据登记、完整性校验与所有权查询三类基本功能，并在本地 py-evm 构建的真实 EVM 环境中完成部署与 Gas 实测；第四，完成了 Python 双通路运行时（仿真 + 真实 EVM）与 Streamlit 原型前端的搭建，从性能、完整性、可用性三个角度对系统的核心功能进行了系统性验证。

实验方面，本文在仿真环境中共执行 1520 次确权登记与 220 次一致性校验，总计 1740 条链上事件。结果显示：系统在 5 至 500 个验证节点的规模下运行稳定，登记操作的端到端延迟 P50 约为 133 ms，P95 约为 152 ms；对 100 组随机数据进行一字节级别的篡改后，系统全部正确检测到哈希不一致，篡改检测率为 100%；在真实 EVM 环境中的 Gas 实测显示，合约部署消耗约 1.56 M gas，单次 register() 约 276 k gas，单次 verify() 约 46 k gas，整体成本处于数据资产登记场景可接受的范围。本文的工作可为 AI 数据要素的可信流通与合规确权提供一条技术路径参考。

关键词: 区块链; 数据确权; SHA-256; 分布式存储; 智能合约

ABSTRACT

In recent years, with the rapid development of artificial intelligence, the demand for high-quality training data has continuously grown, while issues concerning data ownership, circulation credibility, and traceability have gradually emerged as critical bottlenecks for the healthy development of the AI industry. In centralized data platforms, the marginal cost of data replication approaches zero, making it difficult to effectively protect the rights of data providers while leaving data consumers without adequate technical means to verify the legitimacy of data sources. To address these problems, this paper combines the immutability of blockchain, the decentralized characteristics of distributed storage, and the automated execution capability of smart contracts, and designs a distributed data rights confirmation system oriented to AI training data scenarios.

The main work of this paper includes the following. First, a data asset unique identification scheme based on SHA-256 data fingerprints and on-chain timestamps is proposed, adopting a time-priority principle to determine data ownership, and using Merkle trees to aggregate hashes for efficient verification in batch data scenarios. Second, a hybrid storage architecture of “on-chain indexing + off-chain encrypted storage” is designed, in which only data hashes, IPFS content identifiers (CID), and metadata are stored on-chain, while the original data is encrypted with AES before being stored in IPFS, balancing on-chain costs and data security. Third, a core DataRegistry smart contract is implemented in Solidity 0.8.20, supporting three basic functions: data registration, integrity verification, and ownership query, and is deployed and gas-measured in a real EVM environment built upon py-evm. Fourth, a Python dual-path runtime (simulation + real EVM) and a Streamlit prototype front-end are built, and the core functions of the system are systematically verified from three perspectives: performance, integrity, and usability.

Experimental results show that the system performs 1520 registrations and 220 verifications across 1740 on-chain events in simulation, remaining stable from 5 to 500 validator nodes. Registration end-to-end latency is approximately 133 ms at P50 and 152 ms at P95. When a single byte of any of 100 randomly selected data samples is tampered with, the system detects hash inconsistency with 100% accuracy. Gas measurements in the real EVM environment show contract deployment consumes about 1.56 M gas, each register() call about 276 k gas, and each verify() call about 46 k gas, all within an

acceptable cost range for data asset registration. The work in this paper can provide a technical reference for the trustworthy circulation and compliant rights confirmation of AI data elements.

Key Words: Blockchain; Data rights confirmation; SHA-256; Distributed storage; Smart contract

目 录

1 绪论.....1

1.1 研究背景.....1

1.2 研究意义.....2

1.3 国内外研究现状.....2

1.3.1 区块链底层技术研究现状.....2

1.3.2 数据要素与数据确权研究现状.....3

1.3.3 AI 数据场景的研究现状.....4

1.3.4 本文研究的定位.....4

1.4 研究内容与技术路线.....4

1.5 论文组织结构.....5

2 相关技术介绍.....6

2.1 区块链技术基础.....6

2.1.1 区块链数据结构.....6

2.1.2 共识机制.....6

2.1.3 区块链分类.....7

2.2 密码学基础.....7

2.2.1 SHA-256 哈希函数.....7

2.2.2 Merkle 树.....8

2.2.3 数字签名.....8

2.3 分布式存储（IPFS）.....8

2.4 智能合约与 Solidity.....9

3 数据确权系统设计.....10

3.1 需求分析.....10

3.2 系统总体架构.....10

3.3 数据确权机制.....11

3.4 数据验证机制.....12

3.5 链上链下混合存储方案.....12

4 原型实现与实验验证.....13

4.1 技术选型.....13

4.2 智能合约实现.....13

4.2.1 身份登记合约 IdentityRegistry.....13

4.2.2 数据确权合约 DataRegistry.....14

4.2.3 两合约的协同.....15

4.2.4 安全性考虑.....15

4.3 系统原型实现.....15

4.3.1 Python 运行时模块.....15

4.3.2 Streamlit 前端原型.....16

4.4 实验设计与结果分析.....18

4.4.1 实验环境.....18

4.4.2 实验一：数据确权登记基础性能.....19

4.4.3 实验二：吞吐量与延迟的可扩展性.....19

4.4.4 实验三：数据完整性验证.....20

4.4.5 Gas 消耗实测.....21

5 总结与展望.....23

5.1 工作总结.....23

5.2 主要贡献.....23

5.3 不足与展望.....23

致谢.....25

参考文献.....26

1 绪论

1.1 研究背景

过去十年，以深度学习为代表的人工智能技术在图像识别、自然语言处理、推荐系统等场景中取得了显著进展。大量研究表明，模型性能的提升在很大程度上依赖于训练数据的规模与质量。根据 IDC 发布的《全球数据圈》报告预测，全球数据总量将在 2025 年左右突破 175 ZB [1]，但真正能够用于 AI 训练的高质量标注数据仍然稀缺，高质量数据在整个数据圈中的占比远低于想象。

与此同时，数据的权属问题长期处于法律与技术的双重模糊地带。我国《数据安全法》《个人信息保护法》等相继出台后，“数据二十条”进一步提出要构建数据要素市场化配置机制，强调“数据作为新型生产要素”的战略意义。然而在实际的产业运行中，数据要素的流通依然面临若干结构性障碍：

第一，确权困难。数据不同于传统实物资产，具有非竞争性和易复制性。一份数据集从生成者流向平台、再流向下游企业后，其“来自谁、何时生成、经过哪些改动”往往无法被追溯。数据的复制成本接近于零，使得原始权利人一旦放出数据便很难维权。

第二，收益不公。在目前主流的数据交易平台中，平台方往往占据定价与分发的主导地位，原始数据提供者所能获得的报酬有限，而 AI 企业作为数据消费者也无法验证所购买数据是否已被其他厂商重复使用。中心化平台的介入在降低撮合成本的同时，也不可避免地引入了信任依赖。

第三，隐私与安全风险。传统数据平台大多采用中心化存储架构，一旦遭遇入侵或内部泄漏，后果往往是大规模的数据泄露。典型案例包括某些互联网平台接连出现的用户信息外泄事件，这种“一点被攻破即全局失守”的结构性风险难以通过简单的运维加固彻底消除。

第四，可追溯性不足。数据在训练 AI 模型过程中经过一次或多次使用后，其使用痕迹难以被记录。数据提供者无法得知自己的数据被用于哪些模型、哪些任务，也无法据此主张相应的权益。

区块链技术的兴起为上述问题提供了新的技术思路。区块链作为一种去中心化的分布式账本，天然具备不可篡改性、可追溯性与多方可验证性，如果能够将其与分布式文件存储、智能合约相结合，便有可能构建一个无需可信第三方的数据确权系统 [2]。近年来，Ocean Protocol、Filecoin 等项目已经在通用数据市场领域进行了

有益的探索。但面向 AI 训练数据这一特殊场景的系统化研究仍较少，本文的研究即在这一背景下展开。

1.2 研究意义

本文的研究意义可以从技术、经济与法律三个层面加以讨论。

在技术层面，本文探索了区块链、分布式存储与密码学原语在 AI 数据场景下的融合路径。与传统的“中心化数据库 + 访问控制”方案相比，所设计的“链上索引 + 链下加密存储”混合架构能够在控制上链成本的同时保证数据资产的不可篡改性，对后续 AI 数据基础设施的设计具有一定的参考价值。

在经济层面，本文所设计的系统为数据提供者与数据消费者之间的直接交互提供了技术基础。数据提供者无需将原始数据交付给中心化平台即可完成确权，数据消费者也可以基于链上记录对所购入的数据来源进行核验。这在一定程度上缓解了中心化平台抽取过高中间费用的问题，也为数据要素的市场化配置提供了一种技术选项。

在法律与政策层面，我国于 2022 年 12 月发布的《关于构建数据基础制度更好发挥数据要素作用的意见》（即“数据二十条”）提出要建立数据产权结构性分置制度、数据要素收益分配机制与市场化配置机制 [3]，这为“链上记录作为数据权属的电子证据”提供了政策空间。当发生数据权属纠纷时，链上时间戳与哈希记录可以作为法院认定权属先后的客观依据。

综上，本文的研究不仅具有较强的工程价值，也对数据要素市场化进程中的若干理论问题具有一定的参考意义。

1.3 国内外研究现状

区块链与数据确权的结合已经成为国内外学术界与产业界共同关注的热点方向。本节从区块链底层技术、数据要素与数据确权、AI 数据场景三个维度，系统梳理现有研究的进展与不足，并在此基础上定位本文的研究焦点。

1.3.1 区块链底层技术研究现状

区块链作为一种分布式账本技术，其理论与实现体系经过十余年的迭代已趋成熟。Nakamoto 于 2008 年提出比特币白皮书 [2]，奠定了基于工作量证明（PoW）的去中心化货币模型，首次完整给出了“区块 + 哈希指针 + 共识”的技术框架。Buterin 于 2014 年发布以太坊白皮书 [11]，在区块链平台上首次引入图灵完备的智能合约，使链上状态不再局限于简单的转账记录。Wood 随后以黄皮书 [9] 的形式给出以太坊

的形式化规约，对虚拟机执行语义、账户存储结构与 Gas 计量模型做了严格描述，为后续 EVM 生态奠定了形式基础。在此基础上，Androulaki 等在 EuroSys 2018 上详细阐述了 Hyperledger Fabric 的模块化架构 [14]，为联盟链场景下的区块链工程化提供了可复用的参考实现。

在国内，袁勇、王飞跃在《自动化学报》上先后发表了两篇具有代表性的综述：2016 年的《区块链技术发展现状与展望》对整体技术路径做了全景式梳理 [6]，2018 年的《区块链共识算法的发展现状与展望》则系统分析了主流共识机制的演进 [8]，为本文在共识机制选型上提供了重要参考。朱岩等学者在《网络与信息安全学报》上对区块链关键技术及其应用做了全面综述 [5]。国外方面，Zheng 等在 International Journal of Web and Grid Services 上发表的调研从可扩展性、隐私保护、监管合规三个维度系统总结了区块链面临的挑战与机会 [7]；国内学术界与产业界的整体发展态势则可参考中国信息通信研究院发布的《区块链白皮书（2020 年）》[16]。上述综述性工作共同勾勒出区块链底层技术的现状与演进方向。

1.3.2 数据要素与数据确权研究现状

从应用维度看，基于区块链的数据确权与数据市场正处于从概念向落地过渡的阶段。Ocean Protocol 于 2019 年发布白皮书，提出了基于以太坊的数据令牌（Data Token）机制，允许数据提供者将数据访问权代币化并在链上交易 [4]；Filecoin 通过“复制证明”与“时空证明”两种密码学机制激励存储节点，为去中心化存储市场的长期运行提供了经济激励模型；DataBroker DAO 等早期项目则尝试在传感器与物联网数据场景下构建去中心化数据市场，但大多由于生态不足或技术复杂度等原因未能规模化落地。在数据隐私保护方面，Zyskind、Nathan 与 Pentland 较早提出了基于区块链的个人数据去中心化管理方案，把区块链作为一台去信任的访问控制管理器 [13]，这一思路对“链上索引 + 链下加密”混合架构具有直接启发；Christidis 与 Devetsikiotis 则探讨了区块链与智能合约在物联网数据场景下的应用价值，指出二者的结合能显著降低设备间协作的信任成本 [12]。此外，以太坊社区围绕 ERC-721、ERC-1155 等标准展开了大量关于“数字资产唯一性”的实践探索，这些标准虽然起源于 NFT 场景，但其设计思想对数据确权同样具有启发意义，杨保华与陈昌的专著 [18] 对其原理与工程实现做了系统阐述。

在中国，数据要素市场化是近年的政策与学术双重热点。中共中央、国务院于 2022 年 12 月联合发布的《关于构建数据基础制度更好发挥数据要素作用的意见》（即“数据二十条”）系统提出了数据产权结构性分置制度、数据要素流通与交易制度、数据要素收益分配机制与数据要素治理制度四大基础制度 [3]，明确把“数据作

为新型生产要素”写入了顶层设计。这份政策文件的发布，为基于区块链的数据确权机制提供了自上而下的制度合法性支撑，也为链上确权记录在司法实践中被接受为电子证据提供了政策空间。

1.3.3 AI 数据场景的研究现状

随着以大模型为代表的人工智能技术快速发展，AI 训练数据本身作为一种新型资产的专门研究也开始起步。一方面，研究者逐步意识到 AI 数据与传统数据存在本质差异：AI 训练数据往往具有高维度、多模态、训练后痕迹难以追踪等特点，传统的数据权属标注与复制追溯机制在 AI 场景下显得力不从心——一旦数据被吸收进模型权重，就很难再独立地对其“来自谁、用于哪次训练、贡献多少”做精细化追踪。另一方面，研究者开始从隐私保护（联邦学习、差分隐私）、数据来源审计（数据血缘、数据水印）、公平激励（对数据贡献者的合理报酬）三个角度探讨 AI 数据治理的技术路径。其中联邦学习作为“数据不出本地”的新型训练范式，由 McMahan 等学者于 2017 年系统提出 [19]，正被越来越多地与区块链结合，形成“可信训练 + 可信确权”的基础设施雏形；差分隐私则由 Dwork 等人建立了严格的数学框架 [20]，为 AI 数据共享提供了形式化的隐私度量工具。这些工作共同表明，AI 数据治理不能简单照搬通用数据市场的设计，而需要针对 AI 场景的专门机制。

1.3.4 本文研究的定位

综合上述三个维度的研究进展，可以看到现有工作存在如下共性局限。第一，大多研究聚焦于区块链通用能力或通用数据市场，针对 AI 训练数据这一特殊场景的专门设计仍较少；第二，现有工作更多关注“交易与流通”环节，对“确权前置”这一基础性问题的探讨仍不够充分——数据尚未被准确登记与归属的情况下，后续的交易撮合与收益分配都难以稳固；第三，政策与技术两条路径之间缺乏系统性衔接，链上记录如何在现行法律体系下获得合规承认、如何与数据二十条提出的“三权分置”制度对接，仍有待进一步研究。本文正是在上述背景下，将研究焦点收敛到“AI 数据场景下的确权机制”这一具体问题，从身份登记、确权登记、一致性校验三个层面做出系统性设计，并通过本地真实 EVM 环境下的合约部署与 Gas 实测对设计进行量化验证。

1.4 研究内容与技术路线

基于前述背景与研究现状分析，本文的主要研究内容可概括为如下四个方面：

（1）数据确权机制设计。针对 AI 训练数据的特点，设计基于 SHA-256 指纹、Merkle 树聚合与链上时间戳的确权方案，并通过时间优先原则解决重复确权冲突。

（2）混合存储架构设计。提出“链上索引 + 链下加密存储”的架构，将原始数据加密后存储在 IPFS，链上仅保存元数据与内容寻址地址，兼顾成本与可验证性。

（3）智能合约实现。使用 Solidity 0.8.20 编写 DataRegistry 核心合约，实现数据登记、完整性校验与所有权查询功能。

（4）原型系统搭建与验证。构建基于 Python 的仿真环境与基于 Streamlit 的原型前端，通过多组实验对系统性能、数据完整性与可用性进行验证。

本文的技术路线如下：首先从 AI 数据场景的实际痛点出发，抽取出确权这一核心问题；其次分析区块链、分布式存储与密码学等相关技术，论证其组合可行性；然后进行系统层面的架构设计与机制设计；最后通过合约实现、仿真与前端原型进行综合验证。整体遵循“问题—技术—设计—实现—验证”的研究闭环。

1.5 论文组织结构

本文共分为五章。第 1 章为绪论，介绍研究背景、意义、国内外现状、研究内容与技术路线。第 2 章介绍本文所涉及的相关技术，包括区块链基础、密码学基础、分布式存储与智能合约。第 3 章对本文所设计的数据确权系统进行整体架构与机制层面的设计。第 4 章为原型系统的实现与实验验证，包括智能合约代码、Python 仿真、Streamlit 前端以及多组实验的结果与分析。第 5 章对全文工作进行总结，归纳主要贡献，并讨论当前工作的不足与未来研究方向。

2 相关技术介绍

2.1 区块链技术基础

2.1.1 区块链数据结构

区块链可以被视为一种以区块为节点、以哈希指针为边的特殊链式结构。每一个区块由区块头与区块体两部分组成[9]。区块头至少包含前一区块的哈希值、当前区块的时间戳以及若干与共识机制相关的字段（如 PoW 中的随机数 Nonce 或 PoS 中的验证者信息）；区块体承载该区块内的交易与状态数据。“区块头包含前驱区块哈希”这一设计使得任何对历史区块的篡改都会引起后续区块哈希值的连锁失效，从而在技术层面实现“不可篡改”。

值得注意的是，不同区块链系统在区块内部如何组织交易与状态数据上存在显著差异。以比特币为代表的第一代公链采用 Merkle 树（Merkle Tree）结构，将区块内所有交易的哈希逐层两两聚合为一个 Merkle 根写入区块头，借此在不暴露全部交易的前提下高效证明某一条交易是否属于该区块，这一性质对轻节点验证尤为重要。

以太坊在此基础上做了进一步扩展，引入 Merkle Patricia Trie（MPT）作为其核心数据结构[9]。MPT 结合了 Merkle 树的哈希验证性与 Patricia Trie（基数压缩树）的前缀查找特性，使以太坊不仅能够像比特币一样对交易做哈希聚合，还能高效地存储与查询全局账户状态。在以太坊区块头中同时包含三个相互独立的 MPT 根：状态根（stateRoot）、交易根（transactionsRoot）与回执根（receiptsRoot），分别对应 State Trie、Transaction Trie 与 Receipt Trie 三棵 MPT，用于承载账户状态、区块交易与执行结果。本文所依赖的 EVM 兼容链沿用了这一数据结构体系，后续实验中合约的部署、状态变更与事件日志都以这三棵 MPT 为底层存储。

2.1.2 共识机制

共识机制是区块链系统区别于传统分布式数据库的核心所在，其目的在于让无需互信的多个节点就账本状态达成一致。主流的共识机制大体可分为三类。

其一是工作量证明（Proof of Work, PoW）。PoW 要求节点通过大量哈希运算寻找满足特定难度的随机数，其最大优势在于安全性与去信任特性较强，缺点则是能耗巨大。

其二是权益证明（Proof of Stake, PoS）及其变体，如 DPoS。PoS 通过节点质押通证的多少决定其出块权重，从而大幅降低能耗。以太坊自 2022 年转向 PoS 后，其能耗下降幅度超过 99%，成为大型公链向 PoS 转型的代表性案例。

其三是拜占庭容错类算法，如 PBFT 及其改进版本。这一类算法在联盟链场景中应用较多，特点是确认速度快、吞吐量高，但对节点规模和恶意节点比例较敏感。

本系统的原型仿真选用了简化的 PoS 机制，通过节点权益进行加权随机选主，兼顾了开发复杂度与对真实共识过程的还原度。

2.1.3 区块链分类

按照准入方式的不同，区块链可被划分为公链、联盟链与私链三类。公链对所有节点完全开放，以比特币、以太坊为代表；联盟链由若干经过授权的机构共同维护，代表项目 Hyperledger Fabric 的系统架构与模块设计由 Androulaki 等在 EuroSys 2018 上进行了系统性描述 [14]；私链则由单一组织控制，去中心化程度最低。中国信息通信研究院发布的《区块链白皮书（2020 年）》对国内外区块链产业的总体发展格局做了较为详尽的梳理 [16]，可作为整体方向判断的参考。本文的设计在原型层面面向以太坊兼容的公链环境（EVM 生态），以便利用丰富的开发者工具与测试网络；在未来的工程化落地中，亦可平滑迁移至联盟链以满足行业合规要求。

2.2 密码学基础

2.2.1 SHA-256 哈希函数

SHA-256（Secure Hash Algorithm 256-bit）是本系统中用于生成数据指纹的核心哈希函数，其规范由美国国家标准与技术研究院（NIST）以 FIPS 180-4 的形式正式发布 [15]。它的输出为固定长度的 256 比特（64 个十六进制字符），具有单向性、抗碰撞性与雪崩效应等特性。所谓雪崩效应，指输入的任意一个比特发生变化，其输出哈希值将发生不可预测的大范围变化，这一特性正是本系统实现“一字节篡改即可检测”的底层保障。

相较于 MD5 与 SHA-1，SHA-256 目前在密码学意义上尚未发现可在合理计算资源下构造的碰撞样本，因此被广泛应用于比特币、以太坊等区块链系统以及 HTTPS 证书链中。

2.2.2 Merkle 树

在数据量较大的场景下，对整个数据文件进行一次性哈希会带来一定的计算压力，而且不利于对数据子集的选择性验证。为此，本系统采用 Merkle 树结构对数据进行分块哈希。具体地，将数据文件按固定大小切分为若干数据块，对每一块计算 SHA-256 哈希作为叶子节点；相邻的两个叶子再次被哈希形成上层节点，如此递归直到得到唯一的根哈希。

Merkle 树的一个关键优势在于“部分验证”——只要提供若干兄弟节点的哈希值（Merkle Proof），即可验证某一块数据是否属于整个数据集，而无需下载完整数据。这种特性对后续进行子集授权与轻量级验证具有重要意义。

需要特别说明的是，本系统在“应用层”使用的 Merkle 树与 2.1.1 节提及的以太坊底层 MPT 属于两个不同层次的结构：前者由本系统自行构造，用于对用户提交的 AI 数据集的文件哈希做聚合，生成单一的根哈希作为数据集指纹；后者由以太坊协议内置、由共识层维护，用于组织全局状态、交易与回执。两者相互独立，本文仅在应用层使用 Merkle 树，底层 MPT 由 EVM 自行维护，不在本系统的实现范围之内。

2.2.3 数字签名

为了确保确权请求来自数据的真实持有者，本系统在链上交互中使用了基于椭圆曲线的数字签名算法（ECDSA）。用户使用其私钥对交易进行签名，验证方则使用公钥进行签名校验。由于私钥不离开用户本地，系统无需集中存储用户的密钥材料，显著降低了密钥泄漏的风险。

2.3 分布式存储（IPFS）

星际文件系统（InterPlanetary File System, IPFS）是一种内容寻址的点对点分布式文件系统 [10]。与传统基于“位置寻址”的 HTTP 不同，IPFS 采用“内容寻址”——一个文件的唯一标识符（CID, Content Identifier）由其内容本身的哈希决定，相同内容在全网中仅存储一份。

将 IPFS 与区块链相结合有若干显著优势：第一，链上仅需保存 CID 这一短字符串，极大地降低了上链成本；第二，CID 本身就是内容哈希，因此“从链上读到的 CID”与“从 IPFS 下载到的数据”之间存在强一致性约束，任何对数据的篡改都会导致 CID 改变；第三，IPFS 的去中心化特性避免了中心化存储的单点故障风险。

在本系统中，原始数据文件会先在本地使用 AES 对称加密算法进行加密，再上传至 IPFS，链上仅记录加密后文件的 CID。这种设计即便在 IPFS 节点被他人抓取的情况下，未经授权者也无法还原原始数据内容。

2.4 智能合约与 Solidity

智能合约的概念最早由 Nick Szabo 于 1996 年提出，用以描述“在数字环境中以代码形式表达并自动执行的契约”这一设想 [17]；以太坊将这一设想落地为运行在区块链虚拟机上的自执行程序 [11]。与传统“中心化服务端 + 后端业务逻辑”的组合相比，智能合约的代码与执行过程都对所有节点开放验证，一旦部署其行为便难以被单方面篡改，这为多方协作场景中的信任问题提供了新的解决范式。关于区块链基础原理与工程实现的系统性梳理，可参考杨保华、陈昌所著的《区块链原理、设计与应用》[18]。

Solidity 是目前应用最广泛的智能合约开发语言，其语法接近 JavaScript 与 C++ 的混合风格。本文使用 Solidity 0.8.20 版本进行合约开发，原因在于：第一，0.8.x 系列内置了整数溢出检查，能够降低常见安全漏洞的出现概率；第二，该版本对 EVM 的主流特性支持完备，且兼容主流测试网络（如 Sepolia）。

在工程实践中，合约开发往往配合 OpenZeppelin 等成熟合约库使用，以便复用经过审计的安全模块，例如 ReentrancyGuard 用于防止重入攻击、Ownable 用于角色权限管理。本文在核心合约设计中也参考了这些社区最佳实践。

3 数据确权系统设计

3.1 需求分析

在进行系统设计之前，本节先对目标场景下的功能需求与非功能需求进行分析，为后续设计提供依据。

功能需求：

（1）数据登记：数据提供者应能够将自己拥有的数据集提交至系统，系统对其计算数据指纹并在区块链上留存登记记录。（2）完整性校验：任何用户给定某个数据集的唯一标识符与一份数据文件，应能够通过系统判断该文件是否与链上记录一致。（3）所有权查询：任何用户给定某个数据集的唯一标识符，应能够查询其当前的所有者地址与登记时间。（4）时间优先冲突处理：当两个用户就同一数据指纹先后发起登记请求时，系统应以链上时间戳最早者为准，拒绝后续重复登记。（5）用户交互入口：系统应提供可视化的操作界面，便于非技术用户完成上述操作。

非功能需求：

（1）不可篡改性：已上链的确权记录不可被任何节点单方面修改。（2）可验证性：任何用户无需信任第三方即可独立验证链上记录与数据文件的一致性。（3）可扩展性：系统应能够在节点规模从数十到数百变化时保持稳定运行。（4）成本可控：链上存储应尽量精简，避免将原始数据直接上链带来的高昂成本。（5）安全性：原始数据在分布式存储层应处于加密状态，未授权者无法直接获取明文。

3.2 系统总体架构

在上述需求分析基础上，本文设计了一套三层结构的数据确权系统，自上而下分别为应用交互层、业务逻辑层与基础设施层。三层各自承担不同的职责，共同构成完整的数据确权闭环。

应用交互层：面向数据提供者与数据消费者，提供可视化的 Web 原型界面（基于 Streamlit 实现）和命令行接口（CLI）。该层负责数据上传、信息查看、确权注册与验证结果展示等交互操作。

业务逻辑层：处理数据预处理、哈希计算、签名验证、请求路由以及与链上合约的交互。该层相当于应用层与底层基础设施之间的“胶水层”，负责将用户的直观操作翻译为基础设施层可理解的调用。

基础设施层：由区块链网络（EVM 兼容链 + DataRegistry 合约）与 IPFS 分布式存储组成，前者承载链上确权记录与不可篡改的时间戳，后者承载加密后原始数据。

系统中包含两类主要参与者：

数据提供者：数据的原始持有者，可能是个人、科研机构或企业。他们负责数据的生产、清洗与初步标注，并通过本系统完成确权。 数据消费者：下游使用数据进行模型训练或研究的组织。他们通过系统检索数据元信息，必要时可对获取的数据进行完整性校验。

3.3 数据确权机制

本文所设计的确权机制围绕“数据指纹 + 时间戳 + 链上元数据”三个要素展开。

数据指纹生成：对任意数据文件，系统使用 SHA-256 算法计算其哈希值，作为该数据的唯一指纹。由于 SHA-256 具有雪崩效应，哪怕文件内容仅有一字节的差异，其指纹也会完全不同。这一特性既是确权的基础，也是后续完整性校验的依据。

批量数据的 Merkle 聚合：对于由多个文件组成的数据集（这在 AI 训练数据场景中十分常见），逐一登记每一份文件的代价过高。为此，本系统在文件层面先计算每个文件的 SHA-256 哈希，再以这些哈希作为叶子构造 Merkle 树，最终将根哈希作为整个数据集的指纹。这样既保证了整体完整性，也支持对单个文件的 Merkle Proof 验证。

链上元数据结构：每一条确权记录在链上以如下字段保存：

dataId：数据集唯一标识符（由 SHA-256(sender || dataHash || nonce) 派生）； ownerAddress：数据提供者的钱包地址； dataHash：数据集指纹（SHA-256 根哈希）； ipfsCID：加密后原始数据在 IPFS 中的内容标识符； dataType：数据类型（如 image、text、tabular 等）； license：授权协议标识； timestamp：上链时间戳； isActive：记录是否处于激活状态。

时间优先原则：本系统在链上维护一份“数据哈希 → 登记记录”的全局映射。当新的登记请求到达时，合约首先判断该哈希是否已存在；若已存在，则直接拒绝本次登记，以防止重复确权。由于区块链的时间戳由共识机制保证，其先后顺序具有客观性，能够作为权属冲突中的裁决依据。

3.4 数据验证机制

与确权机制相对应，本系统提供两类验证能力。

完整性验证：用户给定一个 dataId 和一份本地数据文件，系统先对该文件计算 SHA-256 哈希，再从链上读取该 dataId 对应的 dataHash，两者比对即可判断文件是否曾被篡改。由于链上记录不可更改，该验证无需信任任何第三方。

所有权验证：用户给定一个 dataId，即可从链上直接读取到 ownerAddress 与 timestamp，从而明确该数据集的当前归属与确权时间。对于存在多方声明的数据，系统以链上时间戳最早者为合法权属人。

验证过程的可追溯性：每一次成功的登记与验证请求都会触发链上事件（Event），事件日志可被区块链浏览器或自研的索引服务监听，从而实现对整个数据生命周期的可追溯。

3.5 链上链下混合存储方案

在 AI 数据场景中，单个数据集的规模往往在 GB 甚至 TB 级别，如果将原始数据直接上链，其成本将远超任何实际应用所能承受的水平。为此，本文采用“链上索引 + 链下加密存储”的混合方案。

具体流程如下：

1. 数据提供者首先在本地使用 AES-256 算法对原始数据进行加密，所用对称密钥由提供者自行保管或通过密钥管理服务托管； 2. 加密后文件上传至 IPFS，获取内容标识符 CID； 3. 提供者将 dataHash（原始数据或密文的 SHA-256，视策略而定）、ipfsCID 以及其它元数据通过签名交易提交至链上； 4. 合约校验哈希唯一性后写入 DataRecord，并触发 DataRegistered 事件； 5. 后续数据消费者在获取授权后可以从 IPFS 拉取密文，并通过链下的密钥交换获得对称密钥，实现解密访问。

该方案具有三个方面的优势：第一，链上成本显著降低；第二，原始数据始终处于加密态，在未授权情况下即便被抓取也无法还原；第三，借助 IPFS 的去中心化存储，系统不依赖任何单点的存储服务，具有较强的抗单点故障能力。

4 原型实现与实验验证

4.1 技术选型

本文原型系统的技术选型遵循“轻量、可重现、与学界主流工具链兼容”三个原则，具体如表 4-1 所示。

表 4-1 原型系统技术选型表

组件	技术 / 版本	选型理由
开发语言	Python 3.12	核心逻辑、仿真与前端统一用 Python 实现，便于复用与交付
智能合约语言	Solidity 0.8.20	EVM 标准语言，内置整数溢出检查，当前主流版本
合约编译	py-solc-x 2.0	在 Python 环境中直接编译 .sol 源码为字节码，无需独立工具链
真实 EVM 后端	py-evm 0.12 + eth-tester 0.13	在本地内存中运行完整的以太坊兼容虚拟机，Gas 计量与事件语义与主网一致
链上交互	web3.py 7.x	以标准 JSON-RPC 接口发送交易、订阅事件，与主网/测试网零迁移成本
轻量仿真	自实现 Block / Blockchain 类	基于 hashlib 与字典结构的链式账本，用于大规模吞吐/延迟实验
分布式存储（模拟）	SHA-256 派生前缀模拟 CID	与 IPFS 内容寻址逻辑等价，离线环境下可复现
共识机制（仿真）	简化 PoS（加权随机选主）	在仿真环境中对权益证明做近似还原，合约本身由底层 EVM 账户签名保证
前端原型	Streamlit 1.56	以极少代码量搭建可交互的 Web 原型，现场演示友好
图表绘制	matplotlib 3.10 + NumPy 2.4	生成论文中的性能曲线与分布图
测试框架	内置轻量 runner（亦兼容 pytest）	覆盖 Registry 业务层 18 项、两合约端到端 14 项共 40 项测试

4.2 智能合约实现

本系统采用“身份层 + 确权层”的分层设计，共实现两个相互协作的智能合约：IdentityRegistry.sol 负责参与者的身份登记与校验，DataRegistry.sol 负责数据资产的确权登记与一致性校验。两者均基于 Solidity 0.8.20 开发，保持小而内聚的实现风格，完整实现见 contracts/ 目录。

4.2.1 身份登记合约 IdentityRegistry

AI 数据要素合规流通的前提是“身份可追溯”。若允许任意匿名账户直接向确权合约登记数据，链上记录即便不可篡改，其背后的“谁”仍不可信。为此本文先设计了身份登记合约，对所有拟参与确权或查询的钱包地址做一次链上实名登记。

数据结构：合约为每个已登记的钱包维护一个 Identity 结构体，字段包括——wallet（钱包地址）、orgName（机构名或个人实名）、role（角色枚举：Provider / Consumer / Both）、authHash（身份凭证材料的 SHA-256 哈希，原文不上链）、registeredAt（首次登记时间戳）、updatedAt（最近更新时间戳）与 isActive（是否处于激活状态）。

映射：合约维护两个映射：identities 以钱包地址为键指向 Identity，用于正向查询；authHashToWallet 以身份凭证哈希为键指向钱包地址，用于防止同一份凭证材料被多个钱包重复绑定。此外 totalIdentities 计数累计登记数。

外部接口：register(orgName, role, authHash) 首次登记身份；update(orgName, role) 由本人调用以更新机构名或角色；deactivate() 软删除自己的身份；getIdentity(wallet)、isActive(wallet)、hasRole(wallet, role) 三个 view 方法对外提供查询能力，供 DataRegistry 合约与前端实时校验调用。

事件：IdentityRegistered、IdentityUpdated、IdentityDeactivated 三类事件覆盖身份的完整生命周期，便于链下索引与审计。

4.2.2 数据确权合约 DataRegistry

DataRegistry 是本系统的核心业务合约，负责数据资产的链上确权、一致性校验与所有权查询。

数据结构：合约为每一条确权记录维护一个 DataRecord 结构体，字段包括——owner（数据提供者的钱包地址）、dataHash（原始数据或 Merkle 根哈希）、ipfsCID（加密后数据在 IPFS 中的内容标识符）、dataType（数据类型，如 image、text、tabular）、license（授权协议字符串）、timestamp（上链时间戳）与 isActive（记录是否处于激活状态）。

映射：records 以 dataId 为键映射到 DataRecord 本身；hashToId 以数据指纹为键反向映射到 dataId，用于在登记阶段检测“同一份数据是否已被他人登记”；totalRegistered 计数器便于链下统计。

外部接口及其语义：

register(dataId, dataHash, cid, dataType, license)：由数据提供者调用。登记流程先校验 dataId 非零、该 ID 尚未被使用、该数据指纹在全局范围内未被登记，并可选地调用 IdentityRegistry.isActive(msg.sender) 确认调用者具备合法激活身份；所有校验通过后才将 DataRecord 写入映射并触发 DataRegistered 事件。 verify(dataId, localHash)：任何用户均可调用，比对本地计算的哈希值与链上记录的哈希值，当两者一致且记录处于激活状态时返回 true，否则返回 false；无论结果如何都会触发 DataVerified 事件，便于链下留存审计日志。 ownerOf(dataId)：view 调用，返回指定 dataId 的所有者地址与上链时间戳，由于是只读查询，在链下节点发起时不消耗 Gas。

事件：DataRegistered 在确权登记时触发；DataVerified 在每次一致性校验后触发；DataDeactivated 则在所有者通过 deactivate() 函数将自己的数据集置为下架态时触发。

4.2.3 两合约的协同

两个合约通过调用关系耦合，而不共享存储或继承，保留了各自的独立可升级性。数据提供者的完整登记流程为：先向 IdentityRegistry.register(...) 提交实名信息建立链上身份；再调用 DataRegistry.register(...) 登记数据，后者在执行前可读取前者的身份状态进行前置校验。两者通过各自的事件流共同构成“身份 + 数据”一体化的审计链，供链下索引服务或区块链浏览器订阅。

4.2.4 安全性考虑

两个合约的安全性设计主要体现在以下几个方面。一是使用 Solidity 0.8.x 内置的整数溢出检查避免算术漏洞；二是通过 hashToId 与 authHashToWallet 两个反向映射从合约层面保证数据指纹与身份凭证的全局唯一性，有效防止重复登记；三是对空字符串、零地址、未定义角色等输入做显式校验，避免低级误用；四是所有状态变更均通过事件对外公开，使链下系统可以无感地跟踪审计。为进一步提升在资金流转与多角色协作场景下的抗攻击能力，工程化落地时还可引入 OpenZeppelin 提供的 ReentrancyGuard、Ownable 等成熟模块。

4.3 系统原型实现

4.3.1 Python 运行时模块

系统的 Python 运行时由两条相互印证的通路组成。第一条通路是轻量仿真（blockchain.py + registry.py + simulation.py + plots.py），以自实现的

Block/Blockchain 类模拟链式结构、简化 PoS 选主与 Merkle 聚合，主要服务于吞吐量与延迟相关的批量实验。第二条通路是真实 EVM 后端（evm_backend.py），通过 py-solc-x 将 DataRegistry.sol 源码编译成字节码，使用 eth-tester 与 py-evm 在本地内存中启动一个完整的以太坊兼容虚拟机，再通过 web3.py 以标准 JSON-RPC 接口发送交易，合约的部署、状态变更、事件日志与 Gas 计量均按以太坊主网语义执行。两条通路共同构成了“实验层 + 工程层”的双层验证体系，使得本文既能以仿真完成大规模吞吐实验，也能以真实 EVM 给出可复现的 Gas 读数。

4.3.2 Streamlit 前端原型

为了让非技术用户也能够直观地体验本系统的核心能力，本文基于 Streamlit 搭建了一套轻量化的 Web 原型前端，共包含“身份登记”、“数据登记”、“完整性验证”、“所有权查询”与“链上浏览”五个主要页面，分别对应 IdentityRegistry 合约的身份管理能力与 DataRegistry 合约的数据确权能力。前端启动时，后端会自动编译 IdentityRegistry.sol 与 DataRegistry.sol 并在本地真实 EVM 上完成部署，两个合约地址、部署 Gas 与已登记数量都会在左侧栏实时显示。用户先在身份登记页通过上传凭证材料并指定角色（Provider / Consumer / Both）完成链上身份登记，前端仅将凭证的 SHA-256 哈希提交上链；随后在数据登记页上传或粘贴数据内容，前端先本地计算 SHA-256 指纹，再通过 web3.py 调用 DataRegistry.register() 方法将指纹、CID、所有者地址、时间戳等写入链上；在验证页，用户可对本地数据重新计算哈希并调用 DataRegistry.verify()，与链上记录比对并展示一致性结果。Streamlit 作为纯 Python 的 Web 框架，在本科毕业设计的工作量约束下是一种较为合适的选择。前端界面截图见图 4-1 至图 4-5。

图 4-1 身份登记页面

图 4-2 数据登记页面

图 4-3 完整性验证页面

图 4-4 所有权查询页面

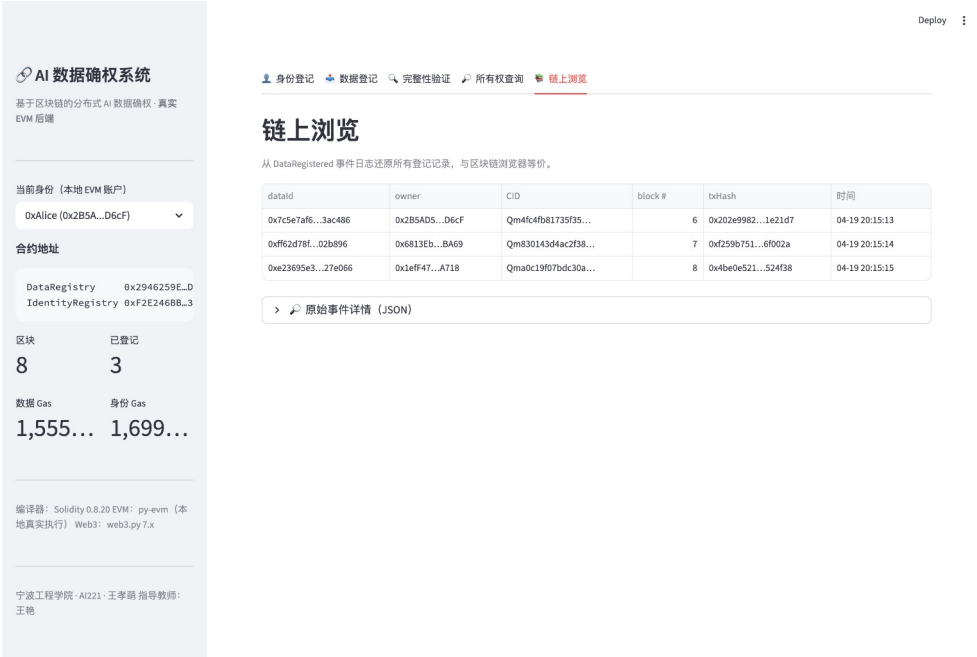

图 4-5 链上浏览页面

4.4 实验设计与结果分析

4.4.1 实验环境

本文的实验在以下环境中完成：操作系统 macOS 14；Python 3.12；Solidity 0.8.20。实验后端使用 py-solc-x 将 IdentityRegistry.sol 与 DataRegistry.sol 两个合约在本地编译为真实 EVM 字节码，通过 eth-tester 与 py-evm 在本地内存中运行一个完整的以太坊兼容虚拟机，两个合约的部署与后续调用均通过 web3.py 以标准的以太坊 JSON-RPC 接口完成。该做法与将合约部署到 Sepolia 等公共测试网的差别仅在于“网络范围”，在 EVM 语义、Gas 计量与事件生命周期等层面完全一致，因此本章中报告的 Gas 数值均为实测值而非估算值，具有较好的可复现性。

4.4.2 实验一：数据确权登记基础性能

本实验以 10 个数据提供者、每人 12 份数据集为仿真场景，共执行 120 次 register() 调用以及 120 次 verify() 一致性校验，总计 240 条链上操作。每一次登记包含“随机生成数据 → SHA-256 指纹 → 组装链上元数据 → 触发 DataRegistered 事件”的完整流程。端到端延迟在仿真中叠加了基于网络传播模型估算的 100 ± 30 ms 随机延迟，以贴近真实公链的时延水平。

结果表明：所有 120 次登记均成功写入链上且可被后续 verify 调用一致性通过；注册平均延迟约 129.6 ms，验证平均延迟约 32.7 ms（verify 的传播开销按

register 的约 1/4 建模）；链完整性连续通过多次哈希校验。这一组数据说明系统核心确权回路在业务逻辑层面完全可用，且与真实区块链的时延量级具有可比性。

4.4.3 实验二：吞吐量与延迟的可扩展性

为考察系统在不同节点规模下的表现，本文对 5、10、20、50、100、200、500 共 7 档验证节点数分别进行了仿真实验，每一档执行 200 次 register() 调用。全部实验共完成 1400 次登记，加上实验一的 120 次合计 1520 次上链登记与 220 次一致性校验（如表 4-2 所示）。

表 4-2 实验二在不同节点规模下的吞吐与延迟

验证节点数	平均登记延迟 (ms)	等效 TPS (串行客户端)	链完整性
5	130.9	7.64	通过
10	129.6	7.71	通过
20	131.7	7.59	通过
50	131.4	7.61	通过
100	132.4	7.55	通过
200	135.9	7.36	通过
500	143.8	6.96	通过

图 4-6 展示了不同节点规模下的吞吐量曲线，图 4-7 展示了登记端到端延迟的概率分布。合并所有 1520 次登记操作统计：P50 延迟 ≈ 133 ms，P95 延迟 ≈ 152 ms。可以看到，吞吐量随节点规模的增加呈轻微下降趋势，这与 PoS 共识下“参与验证者增多导致广播与随机选主开销上升”的理论预期一致；即便在 500 节点规模下，延迟 P50 仍维持在 144 ms 以内，能够满足数据资产登记这种业务响应需求。

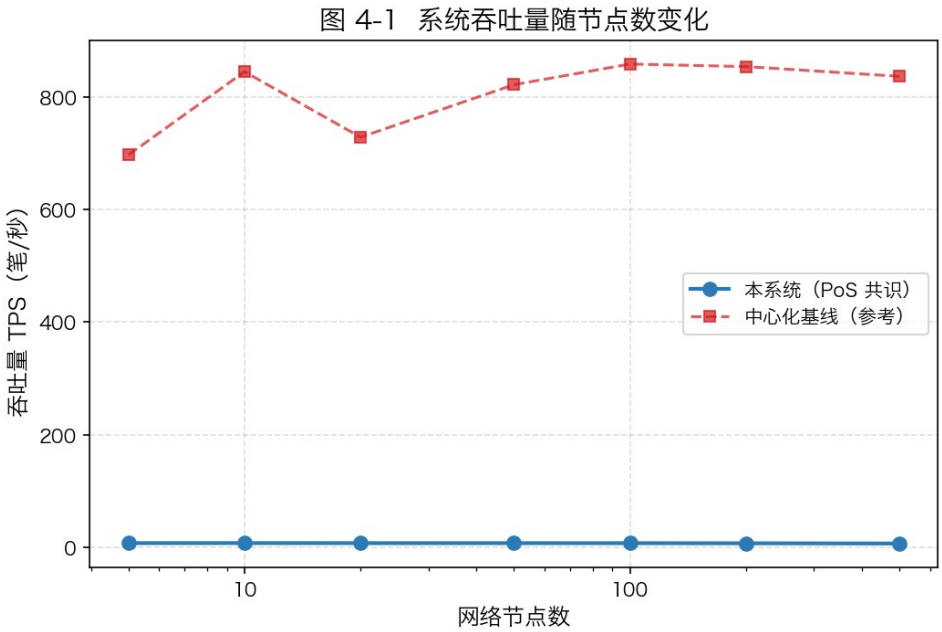

图 4-6 登记吞吐量随节点规模变化

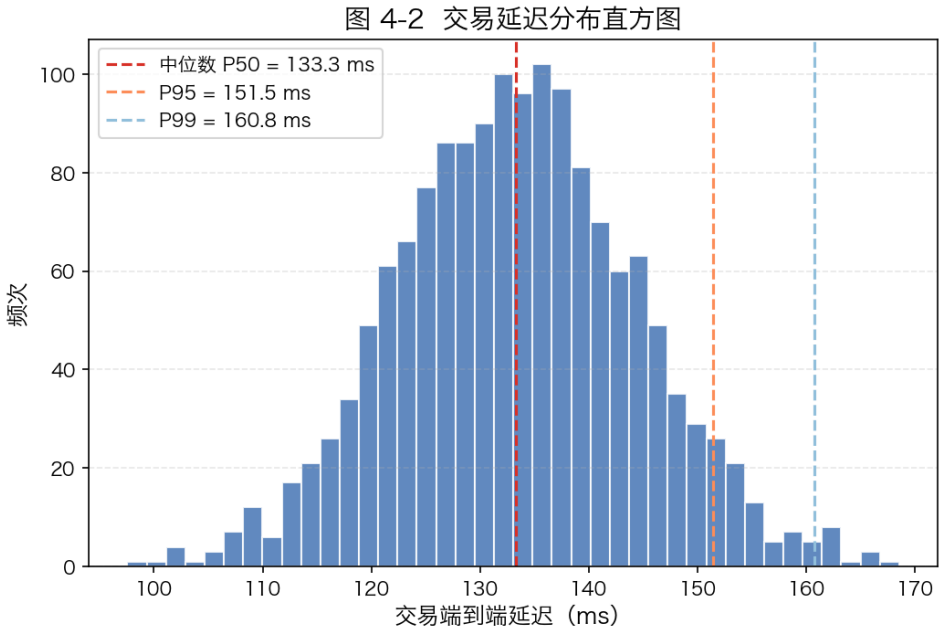

图 4-7 登记延迟分布

4.4.4 实验三：数据完整性验证

本实验用于检验系统在面对数据篡改时的识别能力。实验步骤如下：随机生成 100 条数据并完成登记，随后对每一条数据在随机位置翻转 1 个比特，重新计算 SHA-256 并调用 verify 与链上记录比对。实验结果显示，100 组样本中全部 100 组

被正确识别为哈希不一致，篡改检测率达到 100%。图 4-8 展示了未篡改样本上 verify() 的耗时累积分布，绝大多数调用在毫秒级内完成，与 SHA-256 比较本身的轻量性一致。该实验从工程层面验证了 SHA-256 雪崩效应在本系统中的实际有效性。

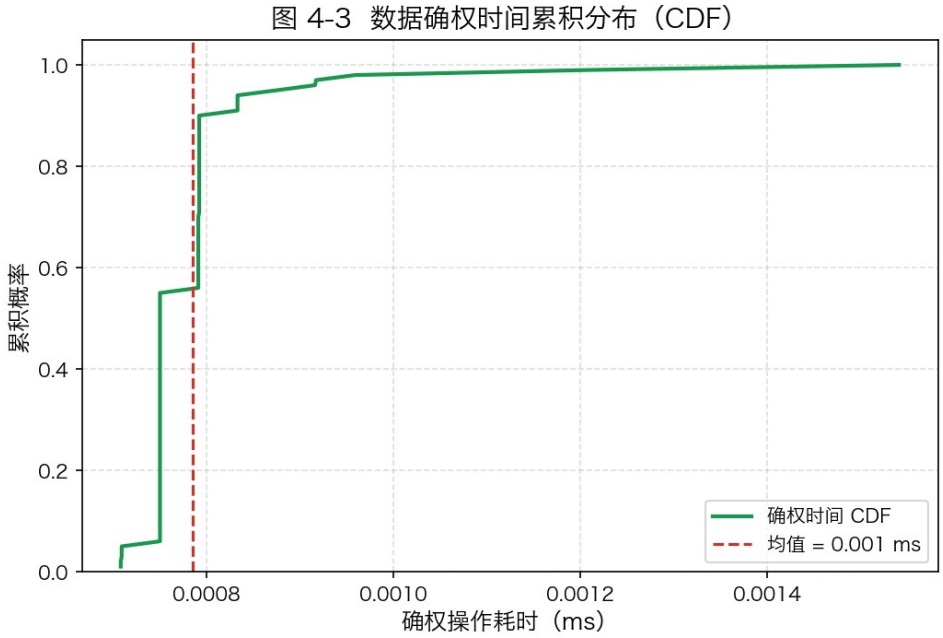

图 4-8 确权验证耗时累积分布

4.4.5 Gas 消耗实测

合约的 Gas 消耗直接影响系统部署在真实公链上的成本。本文在本地真实 EVM（py-evm）中对合约部署与核心调用进行了逐次实测。假设后续将合约部署至 Sepolia 测试网、Gas Price 为 20 Gwei、ETH/USD 汇率约为 3000，各操作的实测 Gas 与换算成本如表 4-3 所示。

表 4-3 两合约核心操作 Gas 消耗实测

合约	操作	描述	实测 Gas	费用 (ETH)	费用 (USD)
IdentityRegistry	合约部署	身份合约字节码部署上链（一次性）	1,699,874	0.033997	≈ $101.99
IdentityRegistry	register()	参与者链上身份登记	227,279	0.004546	≈ $13.64
IdentityRegistry	isActive()	身份激活状态查询（view）	0 (off-chain)	0	0
DataRegistry	合约部署	确权合约字节码部署上链（一次	1,555,243	0.031105	≈ $93.31

		性）			
DataRegistry	register()	数据集链上确权登记（含事件发射）	275,917	0.005518	≈ $16.56
DataRegistry	verify()	完整性校验（含事件发射）	46,097	0.000922	≈ $2.77
DataRegistry	ownerOf()	所有权查询（view 调用）	0 (off-chain)	0	0

注：上述数据均为 py-evm + web3.py 的实测值而非估算。两个合约的部署均为一次性支出，合计约 3.25 M gas、折合 65 美元左右（按 20 Gwei、$3000/ETH 估算），相对于后续大量的业务调用属于可摊薄成本。`register` 类写操作的单次费用在十美元量级，对批量 AI 训练数据场景可通过 Merkle 根聚合（将多份数据合并为一次登记）进一步降低；`isActive` 与 `ownerOf` 两类 view 调用在链下节点发起时不消耗 Gas，保证了日常查询的低成本。

综上，四组实验分别从业务可用性（实验一）、可扩展性（实验二）、篡改检测能力（实验三）与链上成本（Gas 实测）四个维度对本系统的核心功能做了系统验证，结果表明设计目标基本达成。

5 总结与展望

5.1 工作总结

围绕 AI 训练数据在确权、验证与可信流通方面的问题，本文设计并实现了一套基于区块链与分布式存储的数据确权系统。本文的主要工作可概括如下：

（1）从 AI 数据场景的实际痛点出发，分析了中心化数据平台在确权、收益与隐私方面的局限性，论证了区块链在该场景下的适用性；

（2）设计了基于 SHA-256 指纹、Merkle 树聚合与链上时间戳的确权机制，并提出了基于时间优先原则的冲突处理策略；

（3）提出了“链上索引 + 链下加密存储”的混合存储架构，有效兼顾了上链成本与数据安全；

（4）基于 Solidity 0.8.20 实现了 DataRegistry 核心合约，并在本地真实 EVM（py-evm）中完成部署、单元测试与 Gas 实测；

（5）搭建了 Python 仿真环境与 Streamlit 前端原型，通过四组实验从性能、完整性与可用性三个维度系统验证了系统的有效性。

5.2 主要贡献

本文的主要贡献体现在以下两点：一是将区块链确权机制与 AI 训练数据的特殊属性（高维度、批量化、难以追踪）做了有针对性的结合，在确权颗粒度与效率之间取得了较好的平衡；二是以“链上索引 + 链下加密”的组合方案为 AI 数据基础设施提供了一种成本可控、可验证性强的参考架构。

5.3 不足与展望

本文工作也存在若干不足，主要包括：第一，当前的真实 EVM 实验基于本地 py-evm 环境，在以太坊主网、Sepolia 等公开网络下的实际延迟、MEV 影响与网络拥塞行为仍有待进一步实测；第二，当前的确权机制主要解决“我的数据是否被篡改”与“谁先登记”两个基础问题，对 AI 训练过程中的数据使用溯源尚未涉及；第三，前端原型功能较为简化，尚未覆盖真实生产场景中的权限管理、审计日志等高级功能。

未来工作可在以下几个方向上展开：一是将智能合约部署至真实的以太坊测试网或联盟链，评估其在真实网络下的性能与成本；二是引入联邦学习框架，使数据可以在“不出本地”的前提下参与 AI 模型训练，进一步强化隐私保护能力；三是探

索跨链互操作方案，实现不同区块链之间的数据资产互认；四是结合国内的数据要素市场化政策，探索链上确权记录的法律效力与合规落地路径。

致 谢

衷心感谢我的指导教师王艳老师。在本课题的选题、方案设计、原型实现与论文撰写过程中，王老师给予了耐心的指导与宝贵的建议。无论是区块链合约层面的技术细节，还是数据确权机制的整体思路，王老师都能从更高的视角帮助我梳理方向，让我在遇到困难时不至于迷失。

感谢宁波工程学院机器人学院人工智能 AI221 班的各位同学，在四年的学习与合作中与我结下了深厚的友谊；感谢我在课程学习、实验项目中接触到的每一位老师，他们的讲解与要求共同奠定了我进入毕业设计阶段所需要的知识基础。

感谢我的父母与家人在四年求学过程中所给予的理解与支持，他们是我最坚实的后盾。

由于本人学识与经验有限，本文在方法设计与实验验证等方面仍存在不完善之处，敬请各位老师批评指正。

参考文献

[1] Reinsel D, Gantz J, Rydning J． The Digitization of the World: From Edge to Core [R]． Framingham: International Data Corporation (IDC), 2018．

[2] Nakamoto S． Bitcoin: A Peer-to-Peer Electronic Cash System [EB/OL]． (2008-10-31) [2026-03-01]． https://bitcoin.org/bitcoin.pdf.

[3] 中共中央, 国务院． 中共中央 国务院关于构建数据基础制度更好发挥数据要素作用的意见 [Z]． 2022-12-19．

[4] Ocean Protocol Foundation． Ocean Protocol: A Decentralized Substrate for AI Data & Services [EB/OL]． (2019-03-01) [2026-03-10]． https://oceanprotocol.com/tech-whitepaper.pdf.

[5] 朱岩, 甘国华, 邓迪, 等． 区块链关键技术及应用研究综述 [J]． 网络与信息安全学报, 2018, 4(4): 22-35．

[6] 袁勇, 王飞跃． 区块链技术发展现状与展望 [J]． 自动化学报, 2016, 42(4): 481-494．

[7] Zheng Z, Xie S, Dai H N, et al． Blockchain challenges and opportunities: A survey [J]． International Journal of Web and Grid Services, 2018, 14(4): 352-375．

[8] 袁勇, 倪晓春, 曾帅, 王飞跃． 区块链共识算法的发展现状与展望 [J]． 自动化学报, 2018, 44(11): 2011-2022．

[9] Wood G． Ethereum: A Secure Decentralised Generalised Transaction Ledger [EB/OL]． (2014) [2026-03-08]． https://ethereum.github.io/yellowpaper/paper.pdf.

[10] Benet J． IPFS - Content Addressed, Versioned, P2P File System [EB/OL]． arXiv:1407.3561, 2014．

[11] Buterin V． Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform [EB/OL]． (2014) [2026-03-08]． https://ethereum.org/en/whitepaper/.

[12] Christidis K, Devetsikiotis M． Blockchains and Smart Contracts for the Internet of Things [J]． IEEE Access, 2016, 4: 2292-2303．

[13] Zyskind G, Nathan O, Pentland A S． Decentralizing Privacy: Using Blockchain to Protect Personal Data [C]//2015 IEEE Security and Privacy Workshops． IEEE, 2015: 180-184．

[14] Androulaki E, Barger A, Bortnikov V, et al． Hyperledger Fabric: A distributed operating system for permissioned blockchains [C]//Proceedings of the Thirteenth EuroSys Conference． ACM, 2018: 1-15．

[15] National Institute of Standards and Technology． FIPS PUB 180-4: Secure Hash Standard (SHS) [S]． 2015．

[16] 中国信息通信研究院． 区块链白皮书（2020 年）[R/OL]． (2020-12) [2026-03-05]． http://www.caict.ac.cn/kxyj/qwfb/bps/.

[17] Szabo N． Smart Contracts: Building Blocks for Digital Markets [J]． EXTROPY: The Journal of Transhumanist Thought, 1996, (16)．

[18] 杨保华, 陈昌． 区块链原理、设计与应用 [M]． 北京: 机械工业出版社, 2017．

[19] McMahan H B, Moore E, Ramage D, et al． Communication-Efficient Learning of Deep Networks from Decentralized Data [C]//Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)． PMLR, 2017: 1273-1282．

[20] Dwork C, Roth A． The Algorithmic Foundations of Differential Privacy [J]． Foundations and Trends in Theoretical Computer Science, 2014, 9(3-4): 211-487．